the other object. In this snippet from a potential tasks.yml, I'm defining the task as having a relationship with the project defined as projects(:book):

```
chapter:
  title: Write a chapter
  project: book
```

If the relationship is has_many, the multiple values in the relationship can be specified as a comma-delimited list. This is true even if the two objects are in a has_and_belongs_to_many relationship via a join table, although a has_many :through relationship does need to have the join model entry explicitly specified.

Fixture files are also interpreted as ERB (Embedded Ruby) files, which means you can have dynamic attributes like this:

```
runway:
  name: Project Runway
  due_date: <%= 1.month.from_now %>
```

Or you can specify multiple entries dynamically, like this:

```
<% 10.times do |i| %>
task_<%=i%>:
  name: "Task <%= i %>"
<% end %>
```

In the second case, notice that the identifier still needs to be at the leftmost column; you can't indent the inside of the block the way normal Ruby style would suggest. Try not to loop inside fixture files; it gets confusing really quickly, largely because the record names are generated dynamically, making the file hard to read. (The only exception would be if you don't intend to refer to any individual record directly, only via finders; then the naming confusion is less of an issue.) If you find yourself needing dynamic data functionality like this, you're probably better off with a factory tool.

Loading Fixture Data

By default, all your defined fixtures are loaded into the database once at the beginning of your test run. Rails starts a database transaction at the beginning of each test. At the end of each test the transaction is rolled back, restoring the initial state very quickly.

The transactional behavior of fixtures is a problem if you're trying to test transactional behavior in your application. In that case the fixture transaction will overwhelm the transaction you're trying to test. If you need less-aggressive transaction behavior, you can go into the spec/spec_helper.rb file and add the line config.use_transactional_fixtures = false. There's no way to change this behavior